Quicksort Algorithm: Implementation, Analysis, and Randomization

Assignment 5

Peddi Rachana

University of the Cumberlands

MSCS-532: Algorithms and Data Structures

Introduction

Quicksort is a comparison-based divide-and-conquer sorting algorithm. It selects a pivot, partitions the input into values smaller than, equal to, and greater than the pivot, and processes the smaller and larger partitions. This report explains two Python implementations, derives their time and space costs, and analyzes measurements collected from the implemented programs. The experiments compare deterministic last-element pivot selection with random pivot selection across three input sizes and three input distributions (Cormen et al., 2022).

Implementation and Design Choices

Deterministic Quicksort

The deterministic implementation selects the final element of each current subarray as the pivot. It scans the subarray once to form smaller, equal, and larger partitions. Rather than making recursive Python function calls, the current program uses an explicit last-in, first-out stack of pending sorting and emission operations to simulate the recursive partition process. This design avoids Python recursion-depth failures for sorted inputs, but retains the same deterministic pivot rule and asymptotic partition behavior. The result is a newly allocated sorted list the original input remains unchanged.

Randomized Quicksort

The randomized implementation chooses its pivot with random.choice from the current subarray. It recursively sorts the smaller and larger partitions and concatenates those results around the equal partition. Random pivot selection makes the partition structure less dependent on the original input order. The three-way partition also handles repeated values efficiently, because elements equal to the pivot are not repeatedly sorted.

Correctness and Edge Cases

Both implementations preserve every input value, place smaller elements before equal elements and larger elements after them, and stop when a partition contains at most one element. The programs were tested against Python's sorted() result and verified not to modify the original list. Each implementation passed eight test inputs: empty list, singleton, unsorted list, ascending order, descending order, duplicates, negative numbers with zero, and all-equal values (16

successful algorithm–case checks in total). This evidence establishes correctness for the tested cases, not a formal proof for every possible input.

Theoretical Performance Analysis

Best and Average Cases

Partitioning a subarray of length n requires $\Theta(n)$ comparisons and list-building work. If the pivot splits the data into approximately equal partitions, the recurrence is $T(n) = 2T(n/2) + \Theta(n)$. By the Master Theorem, $T(n) = \Theta(n \log n)$. Randomized Quicksort has expected $\Theta(n \log n)$ running time because random pivot selection gives a balanced-enough partitioning process in expectation, even when the original array is sorted (Cormen et al., 2022). Deterministic last-element Quicksort also typically behaves as $\Theta(n \log n)$ on inputs whose pivot ranks are reasonably balanced.

Worst Case

When the chosen pivot is repeatedly the smallest or largest element, one partition contains $n - 1$ elements and the other is empty. The recurrence becomes $T(n) = T(n - 1) + \Theta(n)$, whose sum is $\Theta(n^2)$. Ascending and descending arrays are therefore adverse inputs for the deterministic last-element pivot strategy. Randomized Quicksort still has $\Theta(n^2)$ worst-case time, but consistently selecting extreme pivots is unlikely randomization improves expected performance rather than eliminating the theoretical worst case.

Space Complexity and Overheads

These Python implementations use newly allocated smaller/equal/larger lists and concatenated results they are not in-place Quicksort. Therefore their auxiliary storage cannot be described simply as the $O(\log n)$ stack space of an in-place implementation. Across active partition levels, total temporary allocation is typically $O(n \log n)$ in balanced executions, with $O(n^2)$ total temporary allocation possible in highly unbalanced executions peak live storage depends on evaluation and allocation lifetimes. The deterministic explicit stack avoids Python call-stack overflow, whereas the randomized version additionally uses recursive call frames (expected depth $O(\log n)$, worst depth $O(n)$). Random number generation introduces modest overhead for randomized pivot selection.

Empirical Method

The performance_analysis.py program generated three input distributions—random, sorted, and reverse-sorted-for n = 100, 500, and 1,000. For each combination, it timed both implementations using time.perf_counter(), repeated each timing five times, and reported the median in milliseconds. Before recording a result, it checked that the output matched Python's sorted() function. The recorded results were saved to performance_results.csv. The numbers below were copied from the user's successful VS Code execution they are observed measurements, not theoretical estimates.

Measured Results

Input distribution	n	Deterministic (ms)	Randomized (ms)
Random	100	0.2632	0.2509
Sorted	100	0.4594	0.1708
Reverse sorted	100	0.6338	0.1697
Random	500	0.9823	1.2313
Sorted	500	7.0453	0.5493
Reverse sorted	500	7.4845	0.6121
Random	1000	1.5048	0.9898
Sorted	1000	21.5231	0.8764
Reverse sorted	1000	29.0754	1.0173

Interpretation of Findings

For n = 1,000, deterministic Quicksort took 21.5231 ms on sorted input and 29.0754 ms on reverse-sorted input, compared with 0.8764 ms and 1.0173 ms for randomized Quicksort. Thus, on those two measured cases, randomized Quicksort was approximately 24.6 times and 28.6 times faster, respectively. This large difference is consistent with the $\Theta(n^2)$ behavior of deterministic extreme-pivot partitions and the expected $\Theta(n \log n)$ behavior of randomized partitioning.

On random inputs, the results were closer and not uniformly in favor of randomized Quicksort: at n = 500, deterministic took 0.9823 ms versus 1.2313 ms for randomized. Random pivot selection incurs its own overhead, and measured timings can vary with pivot choices, interpreter scheduling, and memory allocation. A single set of five-repetition medians is therefore illustrative rather than conclusive evidence of universal speed superiority.

Limitations and Reproducibility

The benchmark uses relatively small inputs, a single reported experimental run per input combination, and does not record hardware specifications or a fixed random seed. The two

algorithms also use different control structures (explicit stack versus recursion), which complicates attribution of timing differences solely to pivot choice. A stronger follow-up experiment would repeat complete trials across seeds, record machine and Python versions, test duplicate-heavy inputs, and compare implementations using the same control structure. These limitations do not invalidate the measured results but constrain their generalizability.

Practical Applications

Quicksort illustrates how input distribution and pivot selection can affect runtime and reliability in software systems. The randomized approach can reduce input-order sensitivity in data-processing services, while the deterministic approach demonstrates why worst-case inputs require consideration. In production Python, the built-in sorted() function is generally preferable for routine sorting because it is mature and optimized; these implementations serve the assignment's algorithmic and experimental objectives.

Conclusion

Both Quicksort implementations produced correct results for the tested inputs. The analysis confirms $\Theta(n \log n)$ balanced/expected behavior and $\Theta(n^2)$ worst-case behavior. The empirical results particularly demonstrate how a last-element pivot slows deterministic Quicksort on ordered inputs, while random pivot selection mitigates this pattern in expectation. The assignment also highlights the importance of reporting implementation-specific memory costs, testing edge cases, and interpreting benchmark data cautiously.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Python Software Foundation. (n.d.). time — Time access and conversions. Python 3 documentation. https://docs.python.org/3/library/time.html